

# Web Architectures

Rimbert Rudisch-Sommer

[rimbert.rudisch-sommer@fh-hagenberg.at](mailto:rimbert.rudisch-sommer@fh-hagenberg.at)

# Ziele und Inhalte

- Vertiefung clientseitige Web-Programmierung
  - weiterführende JavaScript Techniken
  - clientseitige Frameworks (Vue.js)
  - HTML-5 APIs (multimedia, webworker, websockets, ...)
  - Web Components
- Vertiefung serverseitige Web-Programmierung
  - Serverseitiges JavaScript (node.js / express.js)
  - PHP Frameworks (Symfony)
- Kennenlernen konkreter Frameworks
- Verstehen der Prinzipien
- Einsatz von KI

# Modus / Ressourcen

- Modus
  - 2 Einheiten VL
  - 2 Einheiten UE
- E-Learning:
  - <https://elearning.fh-ooe.at/course/view.php?id=49091>
  - <https://elearning.fh-ooe.at/course/view.php?id=49089>
- Prüfungsmodus:
  - VL: elektronische Klausur am Ende des Semesters
  - UE: Aufgabenstellungen

# HTML

- Defines Elements
  - basic (html, head, body, ...)
  - structure (header, footer, aside, ...)
  - forms (range, date, ...)
  - media (audio, video, canvas, ...)
- Defines Functionality (DOM / JavaScript APIs)
  - media control (Audio/Video)
  - canvas (bitmap, 3D graphics...)
  - data storage
  - offline application (Caching)
  - web workers (Threading)
  - web sockets (Networking)
  - ...

# (Java|Ecma)Script - History

- 1995: developed by Brendan Eich at Netscape
  - named JavaScript because of the popularity of Java
- 1997: standardized by ECMA (EcmaScript-262)
  - European Computer Manufacturers Association
  - since 1994: Ecma International
- 1998 – 2009: Versions 2, 3 ,(4) – 3.1, 5
- 2015: ES 6 – renamed to ES2015
- From then on: ES2016/17/18/ ... /24/25/26/...
  - ESNext is the name for the upcoming version



# Recap: JavaScript

- Basic Syntax
  - Types & Values: Numbers, Strings, Boolean
  - Expressions: Operators
  - Statements: Selection, Iteration
  - Functions
  - Objects
- DOM
  - `document.getElementById('a')`
  - `document.querySelector('#a')`

# JavaScript Strict Mode

- Strict Mode
  - changes "bad syntax" into real errors
    - results in more secure code
  - variables must be declared
    - variable names cannot be mistyped unnoticed any more
  - duplicating a function parameter is not allowed
  - duplicating an object property is not allowed
  - ...
- Declaration: "use strict";
  - at the beginning of the script: global scope
  - at the beginning of a function: local scope
  - Modules / Classes are automatically in strict mode

```
"use strict";  
let x;  
x = 1;  
y = 2; // This will cause an error
```

# var, let, const

- let

| var                                                  | let                                                          |
|------------------------------------------------------|--------------------------------------------------------------|
| hoisted                                              | not hoisted                                                  |
| always function scope                                | block scope                                                  |
| creates global variable if used outside any function | does not create global variable if used outside any function |

- hoisting: move all variable declarations on top (can be used before declaration)
- use var only if you need to create a global variable

- const

- block scope
- once a const is initialized, its value can never be changed
- if a const references an object, the objects properties can still be changed!
- Object.freeze(o): we cannot change property values, add/delete properties
- Object.seal(o): we can change property values, but cannot add/delete properties

# JavaScript Type System

- number
- string
- boolean
- object
- function
- undefined
  
- operator "typeof"
  - typeof 13 => number
  - typeof "abc" => string
  - typeof [1,2,3] => object

# JavaScript: null vs. undefined

- undefined:
  - variable has been declared, but has not yet been assigned a value
  - is a type by itself (undefined)

```
let testUndefined;  
alert(testUndefined);           // shows undefined  
alert(typeof testUndefined);    // shows undefined
```

- null:
  - null can be assigned to a variable as a representation of "no value"
  - is an object

```
let testNull = null;  
alert(testNull);                // shows null  
alert(typeof testNull);         //shows object
```

- Special Case:  

`null == undefined`

evaluates to true!

# Type Conversion: number

- JavaScript does runtime type conversion if necessary
  - numerical operations: +, -, \*, /
  - string concatenation: + (higher priority than numerical +)
  - comparison operations: ==, !=, >=, <=
- Numerical Operations:
  - if operator is not + or no operand is a string
  - string: convert to number or **NaN**, empty (blank) string is 0
    - 2 \* "2" => 4
    - " " \* 1 => 0 (0\*1)
    - 2 \* "random" => NaN
  - boolean: true => 1, false => 0
    - 2 \* true => 2
    - 2 \* false => 0
  - null => 0
    - 2 \* null => 0
  - undefined => NaN
    - 2 \* undefined => NaN

# Type Conversion: string

- String Contatenation (+)
  - operator + and at least one operand is a string
  - number: convert to string
    - `2 + "abc" => "2abc"`
    - `2 + "2" => "22"`
  - boolean: `true => "true"`, `false => "false"`
    - `"xyz:" + true => "xyz:true"`
    - `false + ":abc" => "false:abc"`
  - null: converted to "null"
    - `"i am " + null => "i am null"`
  - undefined: converted to "undefined"
    - `"i am " + undefined => "i am undefined"`

# Type Conversion: boolean

- Comparison
  - if different types, try to convert both sides to a number
    - "1" == 1 -> 1 == 1 => true
    - "1" == true -> 1 == 1 => true
    - null, undefined convert to NaN => always return false, if compared
    - BUT undefined == null => true
- == tests on equal values, automatic type conversion!
  - true comparisons:
    - 0 == false, "" == false, "0" == false
    - "" == 0, "0" == 0
  - BUT "0" != " "      same type (string) is compared w/o conversion)
- === tests on equal value & type (negation: !==)
  - false === 0 => false
  - undefined === null => false

# Type Conversion: boolean

- boolean context: if (x) ..., while (x) ...
  - number: 0, NaN => false, all else true
  - string: empty string (zero length) => false, nonempty => true
    - if ("" ) => false
    - if (" ") => true !!!! CAVEAT " " == 0 => true, if (0) => false
    - if ("0") => true
  - null, undefined: false
  - ANY object: true

# Objects

- Objects contain
  - properties (values, fields)
    - Can reference nested objects again
  - methods (functions)
- Accessible via dot-notation
  - `<object/variable name>.<property/method name>`  
`document.write("Hello World");`
- Creating objects with `new <class>()`  
`let d= new Date(); // new date object with current timestamp`
- "this" inside a method refers to the object itself

# Predefined Objects

- Math

- Constant object, no calling new ... necessary!
- properties (constant): E, PI, LN2, ...
- methods: sin, cos, floor, ceil, ...

```
document.writeln(Math.sin(Math.PI/2)); -> 1
```

- Date

- methods: getFullYear(), getMonth(), getDate(), getDay(), getHours(), ...

- Number

- constants: MIN\_VALUE, MAX\_VALUE, NaN
- methods: toExponential(), toFixed(), ...

```
let n= 12345.6789;  
n.toFixed(1); // returns 12345.7
```

- String

```
let s1= new String();  
let s2= "This is a String";// string literal also creates object  
s2.length -> 16
```

# Predefined Objects: Arrays

- Creation (new)

```
let a1= new Array();  
let a2= new Array(4,17,"MTD",3.15);  
let a3= new Array(10); // 10 Elements!  
let a4= [2,3,5,7,11] // construction via literal (cf. Strings)
```
- Accessing array elements: [ ]

```
v= a4[0]; a2[0]= 23;  
a4[14]= 47; // dynamically extend arrays by assignment  
           // gaps in the index are allowed
```
- Property **length** (last index+1 cf. strings)

```
a2.length -> 4  
a4.length -> 15 even if only five elements contained
```
- Methods
  - toString(), join(), reverse(), sort(), concat(), slice() ...

# Associative Arrays

- String values are allowed as indices:

```
let color= new Array(),  
color["red"]= "#FF0000";  
color["blue"]= "#0000FF";
```

- Arrays are objects

- Elements can also be accessed via dot notation
- Object properties can be accessed via [] notation

```
let r= color.red
```

- Iterating arrays / object properties:

```
for (let c in color) { ... } // iterate indices  
for (let c of color) { ... } // iterate values
```

# Classes

- Syntax: `class <name> { ... }`
- class names are not hoisted (must be declared before usage)
- automatic strict mode within class code
- everything is public
  - exception: `#` as first character of name indicates private members
- contains declaration of
  - attributes
  - constructor
  - getter/setter
  - methods

# Classes: Examples

```
// Declaration
```

```
class A {  
    #mx;  
  
    constructor() { this.#mx = 0; }  
  
    get x() { return this.#mx; }  
    set x(val) { this.#mx = val }  
  
    hello() { return "my value is " + x; }  
}
```

```
// Usage
```

```
let a = new A();  
console.log(a.x); // 0  
a.x = 42;  
console.log(a.x); // 42  
console.log(a.hello()); // "my value is 42"
```

# Classes

- inheritance works "as usual", keyword **extends**
  - **super** calls superclass
  - If a constructor does not explicitly invoke a superclass constructor, the JavaScript engine automatically inserts a call to the no-argument constructor of the superclass.

```
class B extends A {  
    constructor() { super(); this.mx = -1; }  
  
    hello() { return "hello, my value is " + x; }  
}
```

- **static** methods are executed on the class, no object needed

```
class Util {  
    static inc(x) { return x+1; }  
    static dec(x) { return x-1; }  
}
```

```
console.log(Util.inc(3));
```

# Objects: Constructor functions

- New (kind of) objects can also be defined via constructor functions
  - no classes before ES2015
  - JavaScript classes are only "syntactic sugar" for the mechanism described below

```
// constructor function:
function Rectangle(w, h) {

    // properties
    this.width = w;
    this.height = h;

    // method
    this.calcArea = function Rectangle_calcArea() {
        return this.width*this.height;
    }
}

// Usage:
let rect = new Rectangle(10,7);
rect.width= 5;
rect.calcArea();
```

# Objects

- ways to create objects without classes:
  - literals:

```
let p= {x:5, y:10};
```

- constructor function:

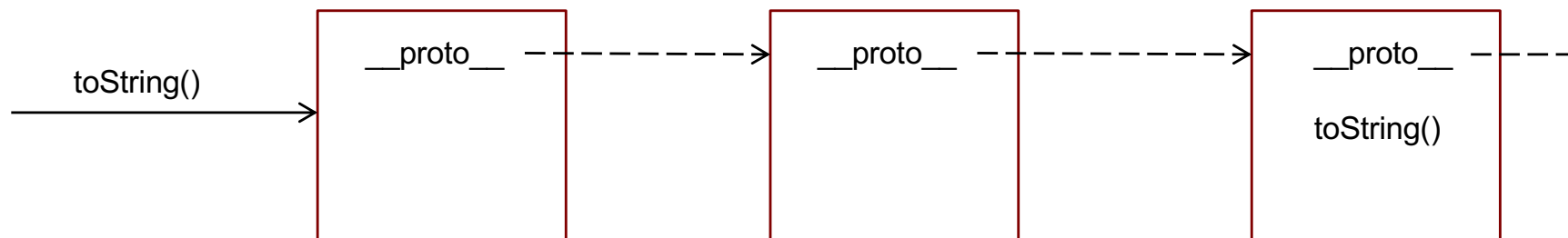
when a function is called together with the operator new, its this variable will reference a new object, that is returned automatically

```
function Circle(radius) {  
    this.radius = radius;  
    this.area = function() { return radius*radius*Math.PI;};  
}
```

```
let c1= new Circle(5);  
c1.area();
```

# Prototypes

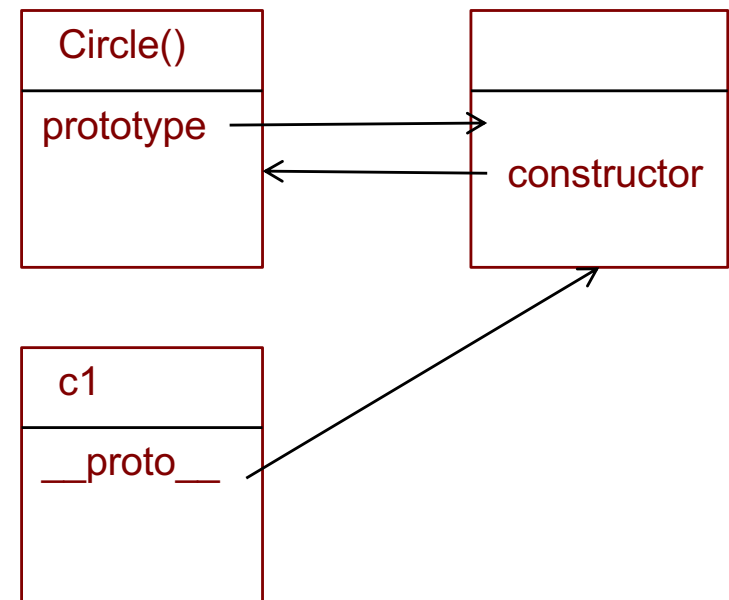
- JavaScript implements the idea of **inheritance** via **prototypes**
  - the implementation of the class keyword still relies internally on this prototype mechanism
- every object refers to a prototype object
  - this reference is a special property,
  - standardwise read-/writeable with `Object.getPrototypeOf()/Object.setPrototypeOf()`
  - "most" implementations (browsers but IE < 11) provide "`__proto__`" (deprecated)
- the prototype object itself refers again to its prototype or null
  - "prototype chain"
- property / method lookup mechanism
  - if a property or method is accessed but not found in an object, the prototype is searched for it,
  - if not found in the prototype, it's prototype is searched ... until null (error)



# Functions & Prototypes

- Objects are created via **constructor functions** + "new"
- Every function (is like an object) has a **standard property "prototype"** referencing an object
- This prototype object has a **property "constructor"** referencing back to the function object
- This prototype object is used as prototype (referenced by `__proto__`) for every object created when the function is called with new

```
function Circle(radius) {  
    ...  
}  
  
c1= new Circle(5);  
  
// c1.constructor => Circle()
```



# Constructor Function vs Literal

- What is the difference?

```
function Circle(radius) {  
  this.radius = radius;  
  this.area = function() { ... };  
}  
  
let c1= new Circle(5);
```

```
function makeCircle(radius) {  
  return {  
    radius: radius,  
    area: function() { ... };  
  }  
}  
  
let c2= makeCircle(5);
```

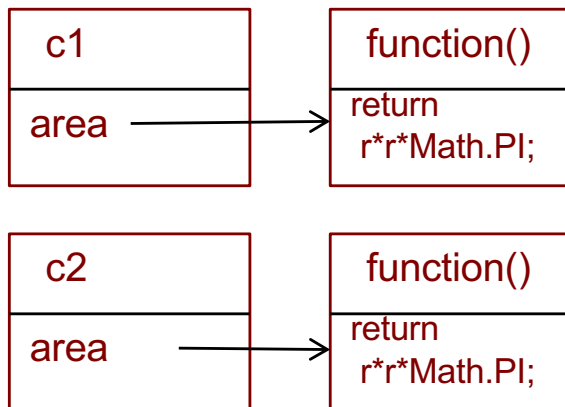
- Objects "inherit" the property "constructor" from their prototype
  - c1.constructor -> function Circle(radius) ...
  - c2.constructor -> function Object() ...
    - "new Object()" is used internally to create object literals { ... }

# Constructor Functions & Methods

- adding methods to a constructor's prototype is more efficient than adding them to "this"
  - function object created only once and found via prototype chain

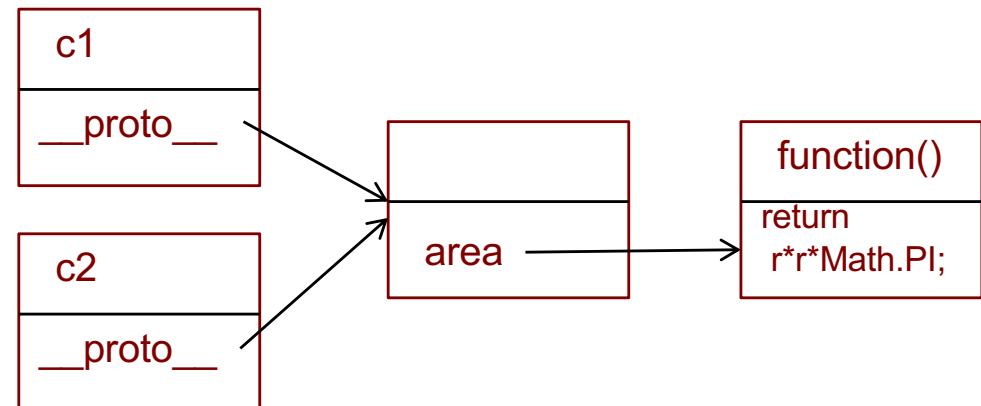
```
function Circle(radius) {  
  this.radius = radius;  
  this.area = function() {...};  
}
```

```
let c1= new Circle(5);  
let c2= new Circle(10);
```



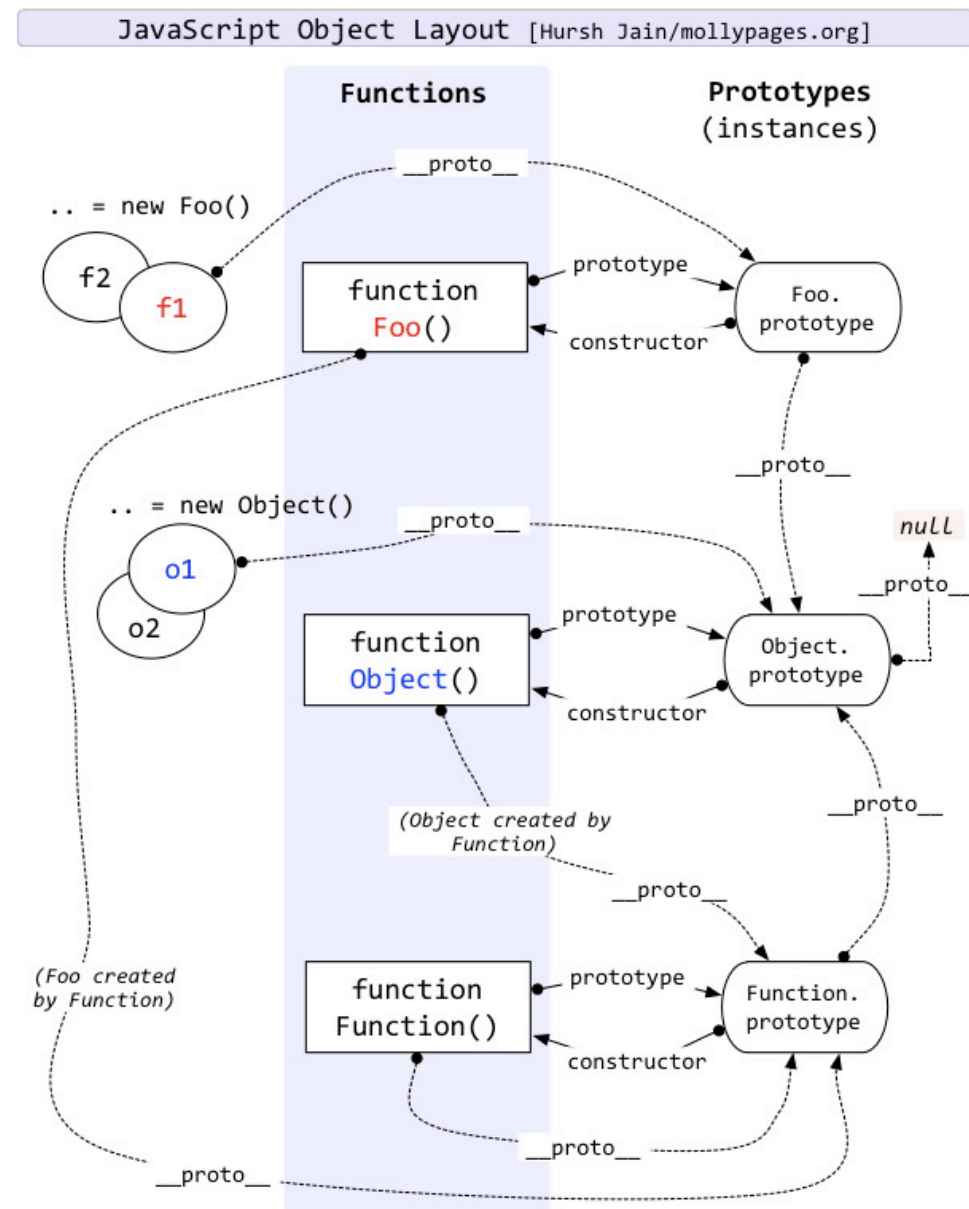
```
function Circle(radius) {  
  this.radius = radius;  
}  
Circle.prototype.area = function() {...};
```

```
let c1 = new Circle(5);  
let c2 = new Circle(10);
```



# JavaScript: Prototypes

- Functions (are objects) and therefore also have a `__proto__` property!!
- also the `Object()` function (constructor for `{}`)
- also the `Function()` function (used to create function objects)



# Mixins

- JavaScript does not allow multiple inheritance
- but the possibility of adding code to prototype
  - `Object.assign(target, source)`  
copies all source properties to target

```
let helloMixin = {  
  sayHello() { alert(`Hello, my name is ${this.name}.`); }  
}
```

```
class User {  
  constructor(name) { this.name = name; }  
}
```

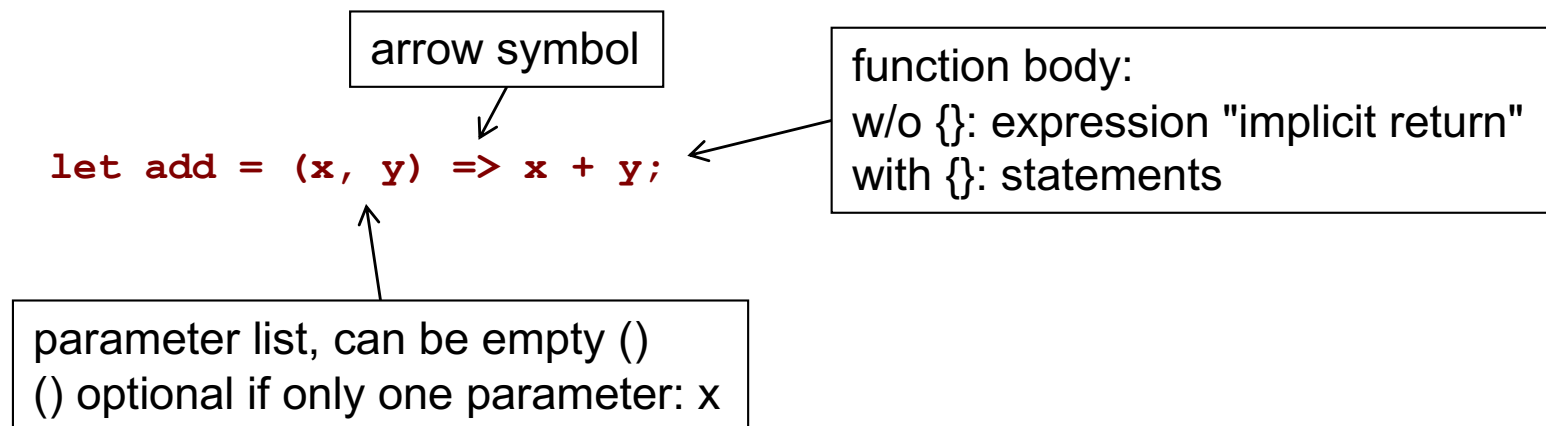
```
Object.assign(User.prototype, helloMixin);
```

```
new User("Bert").sayHello();
```

# Arrow functions

- Alternative syntax for anonymous functions

```
let add = function(x, y) { return x + y }; // old syntax
```



# Local Data Storage

- Mechanism to store data on the client side between page visits
- pre HTML5: Cookies
  - data unit stored by the browser assigned to certain Origins
    - most often used to carry session id
      - sent automatically to assigned server
    - can also be used to store data locally between page visits
  - Cookie JavaScript API is very basic
- HTML5 APIs to store local data
  - Web Storage
    - "improved cookies"
    - 5MB limit (more must be requested by user)
    - supported from all browsers
  - IndexedDB
    - "document data store", "NoSQL-DB"
    - supported from newer browser versions

# Same Origin Policy (SOP)

- browser regulation - introduced for security reasons:
- a script may access only resources from it's own origin
- Origin = Protocol + Host + Port

**http://www.example.com/** != **https://www.example.com/**

**http://www.example.com/** != **http://www.nasa.gov/**

**http://www.example.com/** != **http://www.example.com:8080/**

- HTML5 Storage APIs:  
data is bound to documents of same origin
  - cannot access data from other origins

# Web Storage

- API separates
  - **sessionStorage**
    - only for current window/tab
    - deleted automatically if window/tab closed
    - cp. session cookies
  - **localStorage**
    - bound to origin (Protokol+Host+Port)
    - visible in all pages belonging to this origin
    - must be deleted manually
- similar access API (this example: localStorage)

```
localStorage.superHero= "Herman";  
alert ("Superhero today: "+ localStorage.superHero);  
delete localStorage.superHero;
```
- Caveat: Return type is always String

```
localStorage.test= 15; // automatically typecast to String  
alert (localStorage.test + 5) // results in "155"
```

# Web Storage API

- localStorage and sessionStorage provide the same API

- functions or dot-notation:

- setItem(key, data)                      ...Storage.key= data;
- getItem(key)                              ...Storage.key;
  - returns null, if key non-existent
- removeItem(key)                          delete ...Storage.key;

- additional properties (methods)

- length                                      number of entries
- key(index)                                key of x.th entry
- clear()                                    clear all

- Example: read all entries

```
for (let i=0; i<sessionStorage.length; i++) {  
    let key= sessionStorage.key(i);  
    alert(key+sessionStorage.getItem(key));  
}
```

# Recap: JSON

- JavaScript Object Notation
  - "leightweight" data format
  - "serializing" JavaScript Objects as Strings
  - <http://www.json.org/>
- two Structures:
  - *Objects*: name / value pairs
    - {name1:value1, name2:value2}
  - *Arrays*: lists of values
    - [value1,value2,value3]
  - Names: Strings
  - Values: objects, arrays, literals
  - *Literal*: String, Number, true, false, nil
    - String: character in double qoutes, eg. "abcde"
    - Number: cp. Syntax Javascript, eg. -12.09e-2 , 17

# JSON Example

```
{  
  "firstName": "John",  
  "lastName": "Smith",  
  "age": 25,  
  "address": {  
    "streetAddress": "21 2nd Street",  
    "city": "New York",  
    "state": "NY",  
    "postalCode": "10021" },  
  "phoneNumbers": [  
    { "type": "home", "number": "212 555-1234" },  
    { "type": "fax", "number": "646 555-4567" }  
  ]  
}
```

# Web Storage + JSON

- storing of objects

- JSON API:

- JSON.stringify
- JSON.parse

Object -> JSON String

JSON String -> Object

- Example

- store object

```
let person = {  
  name: 'Herman Munster',  
  age: '33'  
}  
sessionStorage.setItem('person', JSON.stringify(person));
```

- read object

```
let person= JSON.parse(sessionStorage.getItem('person'));  
// JSON.parse(null) results to null
```

# Web Storage Events

- if data is manipulated in one window, all other windows showing pages from the same origin are notified by events
- StorageEvent-object
  - key
  - oldValue
  - newValue
  - url
  - storageArea
- Example

```
function handleStorage(event) {  
    alert("Value of "+event.key+ "changed to: "+event.newValue);  
}  
window.addEventListener('storage', handleStorage);
```